

DATA 260, Homework 3

Rental Housing Listings: Authentication and Retrieval Only RAG

Repo : <https://github.com/akankshashuklaWork/data260-7838>

Value	Result	How it was calculated
SID4	7838	Last four digits of student ID
PORT_BASE	8638	$8000 + (7838 \bmod 900)$
PREFIX	s7838	"s" + SID4
SEED	7838	SID4
VERIFY_SEED	267838	$260000 + \text{SID4}$
DOMAIN_ID	6	$7838 \bmod 8$ (Rental Housing Listings)

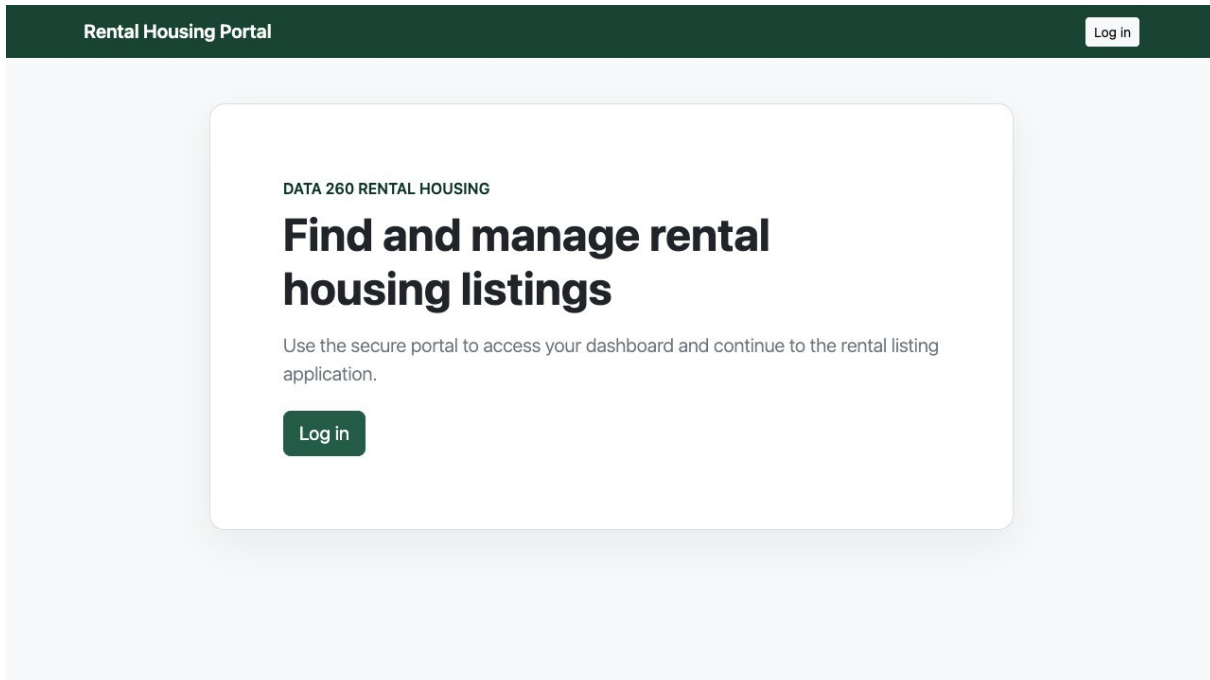
- Hardware: MacBook Pro, Apple M4 Pro, 48 GB RAM
- Python: 3.11.15 in the .venv-hw3 virtual environment
- Local generative model: none. Part 2 is retrieval only, so no model writes or grades any answer.
- Embedding model: sentence-transformers/all-MiniLM-L6-v2 (public, from Hugging Face). It turns text into 384 numbers.
- Tagged commit hash: hw3 (see the repository)
- GitHub repository: <https://github.com/akankshashuklaWork/data260-7838>

Part 1: FastAPI Authentication with Bootstrap

What I built:

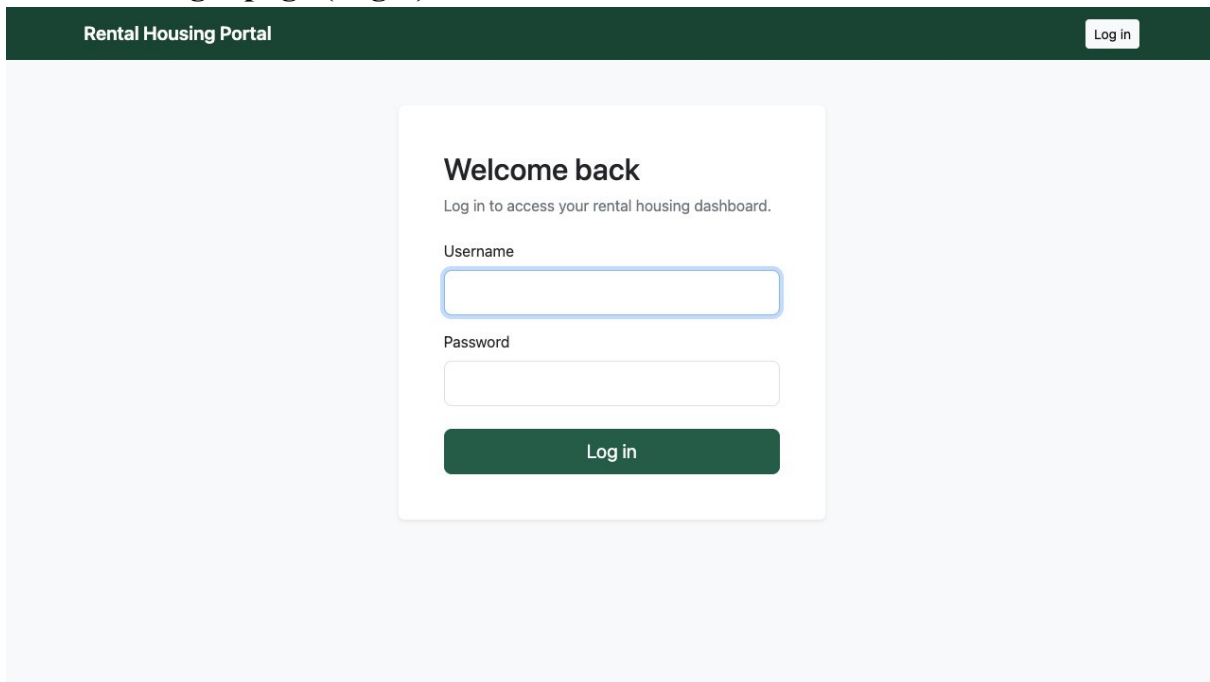
- **Separate router.** All four routes live in code/web_application/auth.py and are attached to the app with include_router.
- **Home page (/).** Shows a welcome message. A signed out visitor sees a Log in link. A signed in user sees Dashboard and Log out links.
- **Login page (/login).** A username and password form. Correct credentials log the user in and redirect to /dashboard. Wrong credentials stay on the page and show a red Bootstrap alert.
- **Dashboard (/dashboard).** Welcomes the user by name. It is protected, so anyone who is not logged in is sent back to /login.
- **Logout (/logout).** Destroys the session and redirects to the home page.
- **Sessions.** Starlette SessionMiddleware (in app.py) stores the logged in user in a signed cookie called s7838_session.
- **Secure cookie.** The cookie is set with Secure, HttpOnly, and SameSite=Lax.
- **Idle timeout.** A session ends after 900 seconds (15 minutes) without a request.
- **Bootstrap.** Navbar for navigation, Cards for the forms and content, Buttons for actions, Alerts for messages. The pages resize for phones.
- **Templates.** base.html, home.html, login.html, and dashboard.html are separate Jinja files.

Screenshot: home page (/)



The public home page. A signed out visitor sees the welcome message and a Log in link.

Screenshot: login page (/login)



The login form is a Bootstrap card with username and password fields.

Screenshot: invalid credentials

The screenshot shows the 'Rental Housing Portal' header with a 'Log in' button. The main content area features a 'Welcome back' message and a login form. A red alert box displays the message: 'Invalid username or password. Please try again.' The form includes fields for 'Username' and 'Password', and a 'Log in' button.

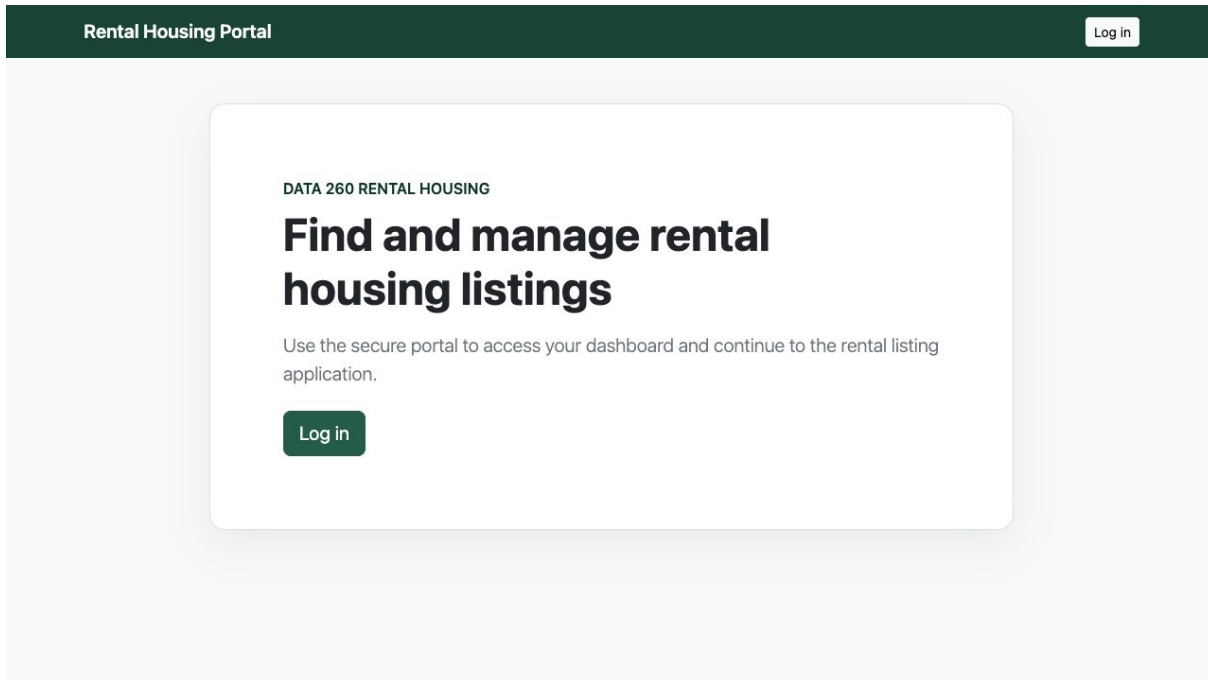
A wrong username or password stays on the login page and shows a Bootstrap alert.

Screenshot: dashboard (/dashboard)

The screenshot shows the 'Rental Housing Portal' header with 'Dashboard' and 'Log out' buttons. The main content area is titled 'PROTECTED DASHBOARD' and welcomes the user 'akanksha'. It includes a session expiration notice: 'You are signed in. Your session expires after 15 minutes of inactivity.' Below this, there are two sections: 'Rental listings' with a 'Manage listings' button, and 'Session controls' with a 'Log out' button.

After a correct login the protected dashboard opens and welcomes the user by name.

Screenshot: logout (/logout)



Clicking Log out clears the session and redirects to the home page. The Log in link is back.

Session cookie settings

- The screenshot below is the real login response header from the running app, captured with curl.
- The set-cookie line shows all three protections: httponly, samesite=lax, and secure.
- Max-Age=3600 is the longest the browser keeps the cookie. The idle timeout is shorter and is checked by the server.
- The signed cookie value is hidden in the screenshot on purpose.

```
curl -i (login response)

$ curl -i -d "username=akanksha&password=****" http://127.0.0.1:8638/login
HTTP/1.1 303 See Other
date: Mon, 21 Sep 2026 08:03:48 GMT
server: uvicorn
content-length: 0
location: /dashboard
vary: Cookie
set-cookie: s7838_session=<signed value hidden>; path=/; Max-Age=3600; httponly; samesite=lax; secure
```

Login response header. The cookie has Secure, HttpOnly, and SameSite=Lax.

Logged out sessions cannot be reused

- First I logged out in the browser and opened /dashboard again. It redirected to /login.
- Then I tested something harder. I copied the session cookie before logging out and tried to use that old copy after logout.
- My first version failed this test. The old cookie still opened the dashboard (HTTP 200), because the whole session lived inside the signed cookie.
- The fix: auth.py now keeps a server side record of live sessions (ACTIVE_SESSIONS). Logout and timeout delete the entry, so a copied cookie is no longer valid.
- After the fix, the old cookie is redirected to /login, as shown below.

Welcome back

Log in to access your rental housing dashboard.

Username

Password

Log in

After logout, opening /dashboard sends the browser back to the login page.

logged-out cookie cannot be reused (server on port 8638)

```
$ curl -m 8 -s -c jar.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" -d "username=akanksha&password=****" http://127.0.0.1:8638/login
HTTP 303 redirect: http://127.0.0.1:8638/dashboard
$ cp jar.txt old.txt

$ curl -m 8 -s -b jar.txt -c jar.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8638/dashboard
HTTP 200 redirect:
$ curl -m 8 -s -b jar.txt -c jar.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8638/logout
HTTP 303 redirect: http://127.0.0.1:8638/
$ curl -m 8 -s -b jar.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8638/dashboard
HTTP 303 redirect: http://127.0.0.1:8638/login
$ curl -m 8 -s -b old.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8638/dashboard
HTTP 303 redirect: http://127.0.0.1:8638/login
```

curl

test. Step 5 replays the cookie copied before logout and is redirected to /login.

Idle timeout

- The real timeout is 900 seconds. To prove it without waiting 15 minutes, I ran a second copy of the app on port 8639 with `SESSION_IDLE_TIMEOUT_SECONDS=3`.
- I logged in, opened the dashboard (200), stayed idle for 5 seconds, and tried again.
- The server redirected to `/login?expired=true`, and the login page showed a yellow expired session alert.

Welcome back

Log in to access your rental housing dashboard.

Your session expired due to inactivity.
Please log in again.

Username

Password

Log in

The login page after the session expired from inactivity.

idle session expires (demo server on port 8639, timeout 3 seconds)

```

$ curl -m 8 -s -c jar2.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" -d "username=akanksha&password=****"
http://127.0.0.1:8639/login
HTTP 303 redirect: http://127.0.0.1:8639/dashboard
$ curl -m 8 -s -b jar2.txt -c jar2.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8639/dashboard
HTTP 200 redirect:
$ sleep 5

$ curl -m 8 -s -b jar2.txt -o /dev/null -w "HTTP %{http_code} redirect: %{redirect_url}\n" http://127.0.0.1:8639/dashboard
HTTP 303 redirect: http://127.0.0.1:8639/login?expired=true

```

curl

test. After 5 idle seconds the dashboard redirects to /login?expired=true.

Templates directory

code/web_application/templates

```

$ ls -la code/web_application/templates
total 32
drwxr-xr-x@ 6 akanksha  staff  192 Sep 18 01:33 .
drwxr-xr-x@ 12 akanksha  staff  384 Sep 18 01:38 ..
-rw-r--r--@ 1 akanksha  staff  1210 Sep 18 01:33 base.html
-rw-r--r--@ 1 akanksha  staff  1375 Sep 18 01:33 dashboard.html
-rw-r--r--@ 1 akanksha  staff   809 Sep 18 01:33 home.html
-rw-r--r--@ 1 akanksha  staff  1616 Sep 18 01:33 login.html

```

The templates folder holds base.html, dashboard.html, home.html, and login.html.

Tests for Part 1

- code/test_auth.py runs 19 checks: public pages, wrong login, correct login, the three cookie settings, the protected dashboard, idle expiry, logout, and the cookie replay check.
- The older HW1 and HW2 browser checks still pass after adding authentication.

```
python code/test_auth.py

$ python code/test_auth.py
PASS: home page loads
PASS: home page shows login link for a signed-out user
PASS: existing listing page remains available
PASS: existing listing API remains available
PASS: dashboard rejects a signed-out user
PASS: dashboard redirects to login
PASS: invalid credentials are rejected
PASS: invalid-login alert is displayed
PASS: valid credentials are accepted
PASS: session cookie has Secure
PASS: session cookie has HttpOnly
PASS: session cookie has SameSite=Lax
PASS: signed-in user can open dashboard
PASS: dashboard displays the user's name
PASS: expired session cannot reuse dashboard
PASS: logout redirects
PASS: logout redirects to home
PASS: logged-out session cannot reuse dashboard
PASS: a session cookie copied before logout cannot be replayed

All HW3 authentication checks passed.
```

All authentication checks pass.

```
node code/web_application/tests/run-tests.js

$ node code/web_application/tests/run-tests.js
PASS: page title is HW1-Akanksha
PASS: largest heading tag names the entity
PASS: primary text input is required and autofocus
PASS: secondary input is required
PASS: submitter email uses email type
PASS: description is a required textarea
PASS: category dropdown has four selectable options
PASS: terms label has required wording
PASS: script is linked at the end of body
PASS: description validation requires more than 25 characters
PASS: terms checkbox is validated
PASS: form data is converted to a JSON string
PASS: JSON string is parsed
PASS: object destructuring extracts title and email
PASS: spread operator adds submissionDate
PASS: submission counter uses a closure
PASS: schema defines the entity fields
PASS: strict mode is enabled
PASS: regular and arrow functions are demonstrated
PASS: Promise and async-await are demonstrated
PASS: successful submissions are added to the page
PASS: delete handler uses bind
PASS: JavaScript parses without syntax errors

All assignment checks passed.
```

The earlier HW1 and HW2 browser checks still pass.

auth.py source code

This is the complete code/web_application/auth.py file.

```
"""Authentication routes for the DATA 260 rental housing application."""
import os
import secrets
import time
from pathlib import Path

from fastapi import APIRouter, Form, Request
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.templating import Jinja2Templates

APP_DIR = Path(__file__).resolve().parent
templates = Jinja2Templates(directory=APP_DIR / "templates")

router = APIRouter()

AUTH_USERNAME = os.getenv("APP_LOGIN_USERNAME", "akanksha")
```

```

AUTH_PASSWORD = os.getenv("APP_LOGIN_PASSWORD", "data260")
SESSION_IDLE_TIMEOUT_SECONDS = int(os.getenv("SESSION_IDLE_TIMEOUT_SECONDS", "900"))

# The signed cookie alone can be copied and replayed, so the server keeps its own # record
of live sessions (id -> last activity). Logout and expiry delete the id. ACTIVE_SESSIONS:
dict[str, float] = {}

```

```

def _active_user(request: Request, *, refresh: bool = True) -> tuple[str | None, bool]:
    user = request.session.get("user")      session_id = request.session.get("sid")
    last_activity = ACTIVE_SESSIONS.get(session_id)    if not user or last_activity is
    None:      request.session.clear()
    return None, False

```

```

    now = time.time()    if now - last_activity >
SESSION_IDLE_TIMEOUT_SECONDS:    ACTIVE_SESSIONS.pop(session_id,
None)    request.session.clear()    return None, True
    if
refresh:
    ACTIVE_SESSIONS[session_id] = now    return
str(user), False

```

```

@router.get("/", response_class=HTMLResponse, name="home") def
home(request: Request) -> HTMLResponse:    user, _ =
_active_user(request)
return templates.TemplateResponse(
request=request,    name="home.html",
context={"user": user},    )

```

```

@router.get("/login", response_class=HTMLResponse, name="login_page") def login_page(request:
Request, expired: bool = False, logged_out: bool = False) -> HTMLResponse:    user,
session_expired = _active_user(request)    if user:    return
RedirectResponse(url="/dashboard", status_code=303)
return templates.TemplateResponse(    request=request,
name="login.html",    context={
        "user": None,
        "invalid_credentials": False,
        "expired": expired or session_expired,
        "logged_out": logged_out,
    },
)

```

```

@router.post("/login", response_class=HTMLResponse, name="login") def
login(    request: Request,    username: str = Form(...),
password: str = Form(...), ) -> HTMLResponse:
    username_matches = secrets.compare_digest(username, AUTH_USERNAME)
password_matches = secrets.compare_digest(password, AUTH_PASSWORD)    if
not (username_matches and password_matches):    return
templates.TemplateResponse(    request=request,
name="login.html",    context={
        "user":
None,
        "invalid_credentials": True,
        "expired": False,
    },
    "logged_out": False,    },
    status_code=401,
)
request.session.clear()
session_id = secrets.token_urlsafe(16)
ACTIVE_SESSIONS[session_id] = time.time()
request.session.update({"user": AUTH_USERNAME, "sid": session_id})
return RedirectResponse(url="/dashboard", status_code=303)

```

```

@router.get("/dashboard", response_class=HTMLResponse, name="dashboard")
def dashboard(request: Request) -> HTMLResponse:
    user, expired =
    _active_user(request)
    if not user:
        target =
        "/login?expired=true" if expired else "/login"
        return
    RedirectResponse(url=target, status_code=303)
    return
    templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={"user": user, "idle_timeout_minutes": SESSION_IDLE_TIMEOUT_SECONDS // 60},
    )
@router.get("/logout", name="logout")
def logout(request: Request)
-> RedirectResponse:
    ACTIVE_SESSIONS.pop(request.session.get("sid"), None)
    request.session.clear()
    return RedirectResponse(url="/", status_code=303)

```

Part 2: Comparing Three LlamaIndex Chunking Techniques

Retrieval only RAG means the program only finds the most relevant pieces of text. It does not use a language model to write an answer. The question is which way of cutting documents into chunks helps the search find the right document.

- Results in this report come only from my rental housing corpus, not from the Tiny Shakespeare warm-up text.

Corpus

- 5 public government PDFs, 10,496,158 bytes in total (the assignment asks for at least 200 KB).
- The program loaded 183 non-empty pages and 504,831 characters of text.
- SOURCES.md lists each source URL and access date (2026-09-18). CORPUS_MANIFEST.json stores each file size and SHA-256 hash. A hash works like a fingerprint, so a changed file is detected.

Local file	Publisher	Bytes	SHA-256 (start)
california_fair_housing_fact_sheet_2022.pdf	California Civil Rights Department	783,534	a914cf469493
california_landlord_tenant_guide_2026.pdf	California Department of Real Estate	3,952,345	ed177b2b2c05
hud_fair_housing_act_guide.pdf	US Dept. of Housing and Urban Development	5,468,378	03467e13022a
san_jose_apartment_rent_ordinance_fact_sheet.pdf	City of San Jose Housing Department	153,264	d6e41d4da7ec
san_jose_tenant_protection_ordinance_fact_sheet.pdf	City of San Jose Housing Department	138,637	90e9a52da93d

Five questions (committed before the results)

- questions.yaml was committed to GitHub before any retrieval results were added, so the expected answers were not changed after seeing the output.
- q1, q2, and q3 depend on facts found in only one document. q4 and q5 are topics other documents also discuss.

ID	Question	Expected answer	Expected source	Only one source
q1	Under the San Jose Apartment Rent Ordinance, what is the maximum regular rent increase and how often may it occur?	5 percent, once during a 12 month period	san_jose_apartment_rent_ordinance_fact_sheet.pdf	Yes
q2	How many just causes for terminating a tenancy are listed under the San Jose Tenant Protection Ordinance?	13 just causes	san_jose_tenant_protection_ordinance_fact_sheet.pdf	Yes

q3	Within how many days after a California tenant moves out must a landlord return the remaining security deposit and provide an itemized statement of deductions?	21 days	california_landlord_tenant_guide_2026.pdf	Yes
ID	Question	Expected answer	Expected source	Only one source
q4	Does California fair housing law protect tenants from discrimination based on source of income, and what advertisement is given as an example?	Yes, and an ad saying No Section 8 is the example	california_fair_housing_fact_sheet_2022.pdf	No
q5	How long after an alleged housing-discrimination violation may a person file a complaint with HUD?	Within one year	hud_fair_housing_act_guide.pdf	No

Setup and settings

- Installed: llama-index, llama-index-embeddings-huggingface, sentence-transformers, faiss-cpu, numpy, pandas (pinned in code/requirements.txt).
- Token chunking: TokenTextSplitter with chunk_size 512 tokens and chunk_overlap 50 tokens. Overlap keeps a sentence from being lost when a cut lands in the middle of it.
- Semantic chunking: SemanticSplitterNodeParser with buffer_size 1 and a 95th percentile breakpoint. It uses the same embedding model to find places where the meaning changes and cuts there.
- Sentence window chunking: SentenceWindowNodeParser with window_size 3. Each sentence is its own chunk, and the neighbouring sentences are saved in the metadata as extra context.
- Each technique gets its own in-memory VectorStoreIndex built on SimpleVectorStore. top_k is 5.

Code excerpts

Chunking: code/hw3_rag/chunkers.py

```
def create_chunkers(embed_model):
    return {
        "token": TokenTextSplitter(
            chunk_size=TOKEN_CHUNK_SIZE,
            chunk_overlap=TOKEN_CHUNK_OVERLAP,
        ),
        "semantic": SemanticSplitterNodeParser(
            embed_model=embed_model,
            buffer_size=SEMANTIC_BUFFER_SIZE,
            breakpoint_percentile_threshold=(
                SEMANTIC_BREAKPOINT_PERCENTILE
            ),
        ),
        "sentence_window": SentenceWindowNodeParser.from_defaults(
            window_size=SENTENCE_WINDOW_SIZE,
            window_metadata_key="window",
            original_text_metadata_key="original_sentence",
        ),
    }
```

```

Indexing: code/hw3_rag/indexing.py def
build_indexes(nodes_by_technique, embed_model):
indexes = {} build_times_ms = {}
    for technique, nodes in nodes_by_technique.items():
print(f"Building {technique} in-memory index...")

        vector_store = SimpleVectorStore()
        storage_context = StorageContext.from_defaults(
vector_store=vector_store
        )
        start_time = perf_counter()

        index = VectorStoreIndex(
nodes=nodes,
        storage_context=storage_context,
embed_model=embed_model, show_progress=True,
        )
        elapsed_ms = (perf_counter() - start_time) *
1000

        indexes[technique] = index
build_times_ms[technique] = elapsed_ms

    return indexes, build_times_ms
Retrieval only helper:
code/hw3_rag/retrieval.py def retrieve_one(index, embed_model,
question, top_k): query_vector = np.asarray(
    embed_model.get_query_embedding(question),
dtype=np.float32,
    )
    retriever =
index.as_retriever(similarity_top_k=top_k)

    start_time = perf_counter()
    retrieved_nodes = retriever.retrieve(question)
latency_ms = (perf_counter() - start_time) * 1000
    results = []
document_vectors = []
    for rank, retrieved in enumerate(retrieved_nodes,
start=1): node = retrieved.node

        # Because metadata was excluded from embeddings, this is chunk text.
embedded_text = node.get_content(
metadata_mode=MetadataMode.EMBED
        )

        document_vector = np.asarray(
            embed_model.get_text_embedding(embedded_text),
dtype=np.float32,
        )
        document_vectors.append(document_vector)
        results.append(
            {
                "rank": rank,
                "source_file": node.metadata.get("source_file"),
                "page_label": node.metadata.get("page_label"),
                "chunk_sha256": sha256(
                    (
                        str(node.metadata.get("source_file", ""))
                        + "\0"
                        + node.text
                    ).encode("utf-8")
                ).hexdigest(),
                "store_score": float(retrieved.score),
                "independent_cosine": cosine_similarity(
query_vector, document_vector,
                ),
                "chunk_length_chars": len(node.text),
                "preview": make_preview(node.text),
                "chunk_text": node.text,
                "window_preview":

```

```

make_preview(
    node.metadata.get("window",
    ""))
    ),
    "window_text": node.metadata.get("window"),
}
)
document_matrix =
np.vstack(document_vectors)
return
{
"latency_ms"
:
latency_ms,
"query_vector_shape": list(query_vector.shape),
"query_embedding_dimension": int(query_vector.shape[0]),
"query_vector_first_8": [
    round(float(value), 6) for value in query_vector[:8]
],
"document_vectors_shape": list(document_matrix.shape),
"results": results,
}

```

Output: corpus check and embedding model

```

python -m hw3_rag.ingestion

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.ingestion
2026-09-21T08:07:10Z
Validated corpus files: 5
Validated corpus bytes: 10496158
Loaded non-empty documents/pages: 183
Extracted characters: 504831
- california_fair_housing_fact_sheet_2022.pdf: 783534 bytes
- california_landlord_tenant_guide_2026.pdf: 3952345 bytes
- hud_fair_housing_act_guide.pdf: 5468378 bytes
- san_jose_apartment_rent_ordinance_fact_sheet.pdf: 153264 bytes
- san_jose_tenant_protection_ordinance_fact_sheet.pdf: 138637 bytes

```

The program checks every PDF against the manifest, then loads the text.

```

python -m hw3_rag.embeddings

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.embeddings
2026-09-21T08:07:13Z
Embedding model: sentence-transformers/all-MiniLM-L6-v2
Test query: What protections are available to rental housing tenants?
Query vector shape: (384,)
Embedding dimension: 384
First 8 values: [0.029001, 0.054515, -0.031529, 0.001382, -0.006053, 0.070992, -0.054769, -0.093578]

```

The embedding model turns a query into a vector of 384 numbers. The first 8 values are shown.

Output: chunk counts

```

python -m hw3_rag.chunkers

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.chunkers
2026-09-21T08:07:17Z

Chunking summary
token: chunks=337, avg_chars=1598.85, min_chars=5, max_chars=3073
semantic: chunks=436, avg_chars=1157.87, min_chars=2, max_chars=4185
sentence_window: chunks=3851, avg_chars=131.09, min_chars=1, max_chars=3769

```

How many chunks each technique made, and how long they are in characters.

Output: retrieval for one shared query

- The shared query is q1 (the San Jose rent increase question).

- Each printout shows the query embedding dimension (384), its first 8 values, the vector shapes ([384] for the query and [5, 384] for the five returned chunks), and a table with rank, store_score, cosine_sim, chunk_len, and a 160 character preview.
- cosine_sim is computed again on my own with NumPy from freshly embedded chunk text. It matches the store score to within 0.0000002.

```
python -m hw3_rag.retrieval

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.retrieval
2026-09-21T08:06:31Z

Controlled retrieval test
Question: Under the San Jose Apartment Rent Ordinance, what is the maximum regular rent increase and how often may it occur?
Expected source: san_jose_apartment_rent_ordinance_fact_sheet.pdf

=== token chunking ===
Query embedding dimension: 384
Query first 8 values: [0.047086, -0.035558, 0.015228, 0.032642, -0.047288, 0.05325, -0.068995, 0.021777]
Query vector shape: [384]
Stacked document vectors shape: [5, 384]
Retrieval latency: 7.12 ms
rank store_score cosine_sim chunk_len preview
1 0.721816 0.721816 1195 Such increases will terminate when additional occupant leaves. (ARO Regulations Section 10.02) •
Parking space fee: Fee cannot exceed $50 a month (ARO Regulatio
source: san_jose_apartment_rent_ordinance_fact_sheet.pdf
2 0.709581 0.709581 1946 APARTMENT RENT ORDINANCE FACT SHEET ABOUT THE ORDINANCE The Apartment Rent Ordinance (ARO) may be
found in San José Municipal Code (SJMC) Chapter 17.23, Parts 1
source: san_jose_apartment_rent_ordinance_fact_sheet.pdf
3 0.669833 0.669833 848 October 1 must make sure tha t notice of the increase is delivered to the tenant personally by
September 1 or mailed to the tenant by August 27. However, a land
source: california_landlord_tenant_guide_2026.pdf
4 0.667065 0.667065 933 is subject to local or state rent stabilization laws and the terms of your rental agreement. For
rental agreements with a fixed term (i.e., 6 months, 1 year, 2
source: california_landlord_tenant_guide_2026.pdf
5 0.661594 0.661595 2408 40 Tenants, however, are not entitled to a credit for rent overcharges paid between March 15,
2019, and January 1, 2020. The law expires on January 1, 2030, unl
source: california_landlord_tenant_guide_2026.pdf
```

Token chunking. Ranks 1 and 2 come from the correct San Jose fact sheet.

```
python -m hw3_rag.retrieval

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.retrieval
2026-09-21T08:06:31Z

Controlled retrieval test
Question: Under the San Jose Apartment Rent Ordinance, what is the maximum regular rent increase and how often may it occur?
Expected source: san_jose_apartment_rent_ordinance_fact_sheet.pdf

=== semantic chunking ===
Query embedding dimension: 384
Query first 8 values: [0.047086, -0.035558, 0.015228, 0.032642, -0.047288, 0.05325, -0.068995, 0.021777]
Query vector shape: [384]
Stacked document vectors shape: [5, 384]
Retrieval latency: 7.60 ms
rank store_score cosine_sim chunk_len preview
1 0.709581 0.709581 2020 APARTMENT RENT ORDINANCE FACT SHEET ABOUT THE ORDINANCE The Apartment Rent Ordinance (ARO) may be
found in San José Municipal Code (SJMC) Chapter 17.23, Parts 1
source: san_jose_apartment_rent_ordinance_fact_sheet.pdf
2 0.674706 0.674706 1006 How often can rent be raised? This depends on whether your rental unit is subject to local or
state rent stabilization laws and the terms of your rental agreeme
source: california_landlord_tenant_guide_2026.pdf
3 0.672892 0.672892 750 Code § 1950.6. The maximum fee is adjusted each year based on changes in the Consumer Price Index
since January 1, 1998. As of December 2023, the maximum allowa
source: california_landlord_tenant_guide_2026.pdf
4 0.661594 0.661595 3126 40 Tenants, however, are not entitled to a credit for rent overcharges paid between March 15,
2019, and January 1, 2020. The law expires on January 1, 2030, unl
source: california_landlord_tenant_guide_2026.pdf
5 0.660664 0.660664 2211 D Rent Stabilization Program City of San José Housing Department 200 East Santa Clara St, 12th
Floor, San José, CA 95113 408-975-4480 • www.sanjoseca.gov/rent •
source: san_jose_tenant_protection_ordinance_fact_sheet.pdf
```

Semantic chunking. The correct fact sheet is at rank 1. Rank 5 is the other San Jose fact sheet.

```
python -m hw3_rag.retrieval

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.retrieval
2026-09-21T08:06:31Z

Controlled retrieval test
Question: Under the San Jose Apartment Rent Ordinance, what is the maximum regular rent increase and how often may it occur?
Expected source: san_jose_apartment_rent_ordinance_fact_sheet.pdf

=== sentence_window chunking ===
Query embedding dimension: 384
Query first 8 values: [0.047086, -0.035558, 0.015228, 0.032642, -0.047288, 0.05325, -0.068995, 0.021777]
Query vector shape: [384]
Stacked document vectors shape: [5, 384]
Retrieval latency: 31.82 ms
rank store_score cosine_sim chunk_len preview
1 0.730907 0.730907 78 Other city ordinances allow a certain percentage increase in rent each year.
source: california_landlord_tenant_guide_2026.pdf
2 0.687210 0.687210 46 RENT INCREASES How much can rent be raised?
source: california_landlord_tenant_guide_2026.pdf
3 0.686644 0.686644 66 92 "California law limits the amount your rent can be increased.
source: california_landlord_tenant_guide_2026.pdf
4 0.677933 0.677933 268 * As of January 1, 2020, most rental units are covered by the Tenant Protection Act which places
a cap on annual rent increases equal to 5% plus inflation, or 1
source: california_landlord_tenant_guide_2026.pdf
5 0.675390 0.675390 171 Even if the rental agreement allows for rent increases, the rental agreement should specify a
limit on how much and how often the rent can be raised (see pages
source: california_landlord_tenant_guide_2026.pdf
```

Sentence window chunking. All five results come from the California guide, so the correct source is missed.

Output: full experiment run and metrics

- Every question was run against every technique (15 runs). Each run is saved as JSON in reports/hw03/raw/retrieval/, with scores, cosine similarities, chunk lengths, and latency.
- code/hw3_rag/metrics.py recomputes the summary tables from those files.

```
python -m hw3_rag.run_experiment

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.run_experiment
2026-09-21T08:04:44Z
2026-09-21 01:04:52,644 - INFO - Loading SentenceTransformer model from sentence-transformers/all-MiniLM-L6-v2.
Creating token chunks...
Creating semantic chunks...
Creating sentence_window chunks...
Building token in-memory index...
Building semantic in-memory index...
Building sentence_window in-memory index...
Running q1: Under the San Jose Apartment Rent Ordinance, what is the maximum regular rent increase and how often may it occur?
token: topl=san_jose_apartment_rent_ordinance_fact_sheet.pdf, score=0.721816, latency=7.46 ms
semantic: topl=san_jose_apartment_rent_ordinance_fact_sheet.pdf, score=0.709581, latency=7.90 ms
sentence_window: topl=california_landlord_tenant_guide_2026.pdf, score=0.730907, latency=32.18 ms
Running q2: How many just causes for terminating a tenancy are listed under the San Jose Tenant Protection Ordinance?
token: topl=san_jose_tenant_protection_ordinance_fact_sheet.pdf, score=0.751856, latency=6.61 ms
semantic: topl=san_jose_tenant_protection_ordinance_fact_sheet.pdf, score=0.749611, latency=7.18 ms
sentence_window: topl=san_jose_tenant_protection_ordinance_fact_sheet.pdf, score=0.722435, latency=31.53 ms
Running q3: Within how many days after a California tenant moves out must a landlord return the remaining security deposit and provide an
itemized statement of deductions?
token: topl=california_landlord_tenant_guide_2026.pdf, score=0.796959, latency=6.63 ms
semantic: topl=california_landlord_tenant_guide_2026.pdf, score=0.734426, latency=7.18 ms
sentence_window: topl=california_landlord_tenant_guide_2026.pdf, score=0.760806, latency=32.73 ms
Running q4: Does California fair housing law protect tenants from discrimination based on source of income, and what advertisement is given
as an example?
token: topl=california_landlord_tenant_guide_2026.pdf, score=0.676165, latency=6.67 ms
semantic: topl=california_landlord_tenant_guide_2026.pdf, score=0.747219, latency=7.50 ms
sentence_window: topl=california_landlord_tenant_guide_2026.pdf, score=0.791594, latency=32.97 ms
Running q5: How long after an alleged housing-discrimination violation may a person file a complaint with HUD?
token: topl=hud_fair_housing_act_guide.pdf, score=0.742388, latency=6.31 ms
semantic: topl=hud_fair_housing_act_guide.pdf, score=0.802486, latency=7.01 ms
sentence_window: topl=hud_fair_housing_act_guide.pdf, score=0.857518, latency=31.20 ms
Saved 15 raw retrieval files under /Users/akanksha/Documents/HW2/data260-7838/reports/hw03/raw/retrieval
```

The experiment printed the top result for each question and technique.

```
python -m hw3_rag.metrics

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.metrics
2026-09-21T08:05:59Z

Aggregate retrieval metrics
token: chunks=337, avg_chars=1598.85, top1_cos=0.737837, mean@k=0.685544, recall@k=1.00, top1_accuracy=0.80, latency_ms=6.74
semantic: chunks=436, avg_chars=1157.87, top1_cos=0.748664, mean@k=0.685382, recall@k=1.00, top1_accuracy=0.80, latency_ms=7.35
sentence_window: chunks=3851, avg_chars=131.09, top1_cos=0.772652, mean@k=0.714652, recall@k=0.60, top1_accuracy=0.60, latency_ms=32.12
```

metrics.py recomputed the summary numbers from the saved raw files.

```
python -m hw3_rag.verify

$ HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 PYTHONPATH=code python -m hw3_rag.verify
2026-09-21T08:06:03Z
PASS: 166 checks; maximum score delta=1.82871652821e-07
Wrote /Users/akanksha/Documents/HW2/data260-7838/reports/hw03/verification.json
```

The self-check script passed 166 checks and wrote verification.json.

Results

- Top-1 cosine is the highest similarity among the top 5, averaged over the five questions.
- Mean@5 cosine is the average of all five similarities, averaged over the questions.
- Recall@5 is 1 for a question when the expected PDF appears anywhere in the top 5, otherwise 0, averaged over the questions.
- Latency is the time the similarity search took. Values are rounded to 4 decimals here; METRICS.md has more.

Technique	Chunks	Avg chunk length (chars)	Top-1 cosine	Mean@5 cosine	Recall@5	Mean retrieval latency (ms)
Token	337	1598.85	0.7378	0.6855	1.00	6.74
Semantic	436	1157.87	0.7487	0.6854	1.00	7.35
Sentence window	3851	131.09	0.7727	0.7147	0.60	32.12

- Rank 1 came from the expected source for 4 of 5 questions with token chunking, 4 of 5 with semantic chunking, and 3 of 5 with sentence window chunking.

Per question results:

Technique	Question	Expected source first rank	Recall@5	Top-1 cosine	Mean@5 cosine	Latency (ms)
Token	q1	1	1	0.7218	0.6860	7.46
Token	q2	1	1	0.7519	0.6781	6.61
Token	q3	1	1	0.7970	0.7057	6.63
Token	q4	2	1	0.6762	0.6392	6.67
Token	q5	1	1	0.7424	0.7187	6.31
Semantic	q1	1	1	0.7096	0.6759	7.90
Semantic	q2	1	1	0.7496	0.6510	7.18
Semantic	q3	1	1	0.7344	0.6886	7.18
Semantic	q4	3	1	0.7472	0.6668	7.50

Semantic	q5	1	1	0.8025	0.7446	7.01
Sentence window	q1	miss	0	0.7309	0.6916	32.18
Sentence window	q2	1	1	0.7224	0.6812	31.53
Technique	Question	Expected source first rank	Recall@5	Top-1 cosine	Mean@5 cosine	Latency (ms)
Sentence window	q3	1	1	0.7608	0.7299	32.73
Sentence window	q4	miss	0	0.7916	0.7094	32.97
Sentence window	q5	1	1	0.8575	0.7611	31.20

A confident result that does not contain the answer

- Question q1 with sentence window chunking. Rank 1 had a cosine of 0.7309, which is higher than the best token score (0.7218) for the same question.
- The chunk was one sentence from the California guide: "Other city ordinances allow a certain percentage increase in rent each year."
- It does not answer the question. The right answer is 5 percent, once in 12 months, and it lives in the San Jose Apartment Rent Ordinance fact sheet. That PDF did not appear anywhere in the top 5.
- Why the embedding thought it was similar: the sentence uses the same everyday words as the question (city ordinance, percentage, increase in rent). The model measures meaning and topic, not whether a specific number is present.
- Second example: for q4, token chunking ranked a California guide chunk about discriminatory advertising first (cosine 0.6762). It talks about ads and discrimination but not about source of income, so it looks similar without holding the answer.
- Lesson: a high cosine score means the topic is close. It does not prove the answer is inside the chunk.

Observations

- Token and semantic chunking both found the expected PDF in the top 5 for all five questions (Recall@5 of 1.00) and at rank 1 for four of five.
- Why they worked: their chunks are long enough (about 1,600 and 1,160 characters) to hold a full fact together with its context, such as the ordinance name next to the 5 percent number.
- Semantic chunking made more and shorter chunks than token chunking (436 versus 337) but kept the same source recall. Its cuts follow changes in meaning instead of a fixed size.
- Sentence window chunking made 3,851 tiny chunks (about 131 characters each). A single sentence often lacks the words that identify its document, so generic sentences from a big guide beat the specific fact sheet. Its Recall@5 dropped to 0.60.
- Sentence window also had the highest cosine values (0.7727 top-1) but the worst recall. Short generic sentences score high on similarity without being the right source.
- Sentence window was also about 4 to 5 times slower (32.12 ms versus 6.74 and 7.35 ms) because it searches many more chunks.
- I did not run extra queries, so there is no different best technique to report.

Conclusion

- For this rental housing corpus, I judge semantic chunking the best, with token chunking a very close second.
- Both found the right source for every question (Recall@5 of 1.00). Semantic had the higher top-1 cosine (0.7487 versus 0.7378) and shorter, cleaner chunks. Token was slightly faster (6.74 ms versus 7.35 ms).
- Sentence window was the weakest choice here (Recall@5 of 0.60 and the slowest search), even though its similarity scores looked the highest.
- With only five questions, these results show a trend for this corpus and should not be treated as a general rule.

AI Use

- **1. What I used AI for, and what I did myself.** I used OpenAI Codex to help me understand the HW3 requirements, explain terminal commands, suggest a structure for the authentication and retrieval code, troubleshoot dependency errors, review saved experiment files, and organize the report in beginner friendly language. I ran the commands on my own computer, chose and downloaded the five public PDFs, checked their sources, wrote and reviewed the five questions, inspected the retrieval output, tested the app, and decided what evidence to include. I reviewed all AI assisted code and writing before keeping it.
- **2. AI output that was wrong or unsuitable.** An earlier AI assisted version of the logout code redirected to `/login?logged_out=true`. The assignment says logout must destroy the session and redirect to the home page.
- **3. How I detected it.** I compared the code with the assignment requirement, inspected the redirect the logout route returned, and used the authentication test to check the Location header.
- **4. What I changed and why it works now.** The logout route in `auth.py` now returns `RedirectResponse(url="/", status_code=303)` after clearing the session. The test now expects `/`, and all checks pass.

Reproduce and verify

- Repository: <https://github.com/akankshashuklaWork/data260-7838>, tag `hw3`.
- `reports/hw03` holds `RUN_LOG.txt`, `raw/`, `METRICS.md`, `AI_USE.md`, `verification.json`, `SOURCES.md`, `CORPUS_MANIFEST.json`, `questions.yaml`, and the run instructions.
- Run the app: `cd code`, then `uvicorn web_application.app:app --port 8638`. Run the checks: `python code/test_auth.py`, then `PYTHONPATH=code python -m hw3_rag.run_experiment, hw3_rag.metrics, and hw3_rag.verify`.